

When the Guard Adds Nothing: A Preregistered Paired Evaluation of Generate–Validate–Repair for LLM-Produced Vendor CLI Configurations

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory experiment to measure whether a deterministic generate–validate–repair (G-V-R) pipeline increases the probability that an LLM-produced vendor CLI configuration passes a conservative deterministic validator, relative to direct LLM generation. The preregistered design frozen a single shared initial candidate per task (one generator call shared by both arms) and permitted at most one automatic repair call for the guarded arm if the shared initial failed. Using a curated 300-task multi-vendor manifest and the declared generator snapshot (gpt-5-mini; execution/model_configuration.json SHA-256 = 1acce92558edeb13cd97b75817329d332afe4a36d01d566f1a26c61b132923fab), every shared initial candidate passed the deterministic validator: baseline = 300/300; guarded = 300/300. The paired contingency table is $n_{11} = 300$, $n_{10} = n_{01} = n_{00} = 0$. Exact McNemar two-sided $p = 1.00$; paired bootstrap percentile 95% CI = [0.00, 0.00] (10,000 resamples, seed = 1729). Because no initial failures occurred, the preregistered single-repair policy was never invoked. We release the full preregistration, task manifest, raw responses, provider traces, validator traces, and analysis artifacts with canonical SHA-256 hashes to enable exact reproduction and controlled follow-ups.

I. INTRODUCTION

Generative language models are being proposed to convert high-level operator intent into low-level vendor CLI configurations to accelerate NetOps. Practical deployment demands operational trust: generated changes must satisfy syntactic rules, resource constraints, and workflow safety invariants before being applied. A widely recommended architectural pattern is generate–validate–repair (G-V-R): a generator proposes a candidate configuration, a deterministic validator checks a set of conservative safety invariants, and an automatic repair step attempts bounded corrections when the validator fails. Tool-assisted self-correction and verifier-anchored guardrails are recurring recommendations in recent literature, but quantitative assessments of the marginal benefit of a bounded repair stage—holding generator snapshot, prompt, and validator fixed—are limited.

Research question. Under a shared-initial paired design (one generator call per task, shared by both arms), does allowing a single automatic repair call in the guarded arm increase the per-task probability of producing a deterministic-validator-passing configuration compared to direct generation (baseline)?

Contributions. (1) A machine-readable preregistration that froze experimental design, analysis, and stopping

rules (study_hosted_netops_gvr_v1_cnd_7a3f2b1e; preregistration SHA-256 = a85d66d4c6ce2275e2d8c820f96c610532dbb5452bebb3fe108848f9dc38cd) (2) A curated 300-task multi-vendor intent manifest and a conservative deterministic validator (deterministic_netops_validator_v5), both archived with canonical hashes (task_manifest.jsonl SHA-256 = 6d99060659a722e6a91dd2abd7baca45a2aefb41e124b049124d5ea2632cb4) (3) A preregistered paired analysis executed exactly as specified: $n = 300$ pairs, exact McNemar two-sided test and paired bootstrap CI (10,000 resamples, seed = 1729). (4) A fully archived execution bundle (raw responses, provider traces, validator traces, and analysis artifacts) with SHA-256 hashes to enable exact reproduction (execution/execution_manifest.json lists all artifacts).

II. RELATED WORK

LLM-assisted intent translation and intent-based networking have seen rapid growth. Representative verified works that motivated design choices include Nguyen et al., 2024 (intent translation with LLMs; DOI: 10.1002/nem.2313) and recent CN S M/NetOps demonstrations of LLM-driven configuration generation (e.g., Chakraborty et al., CNSM 2024). Tool-assisted self-refinement and constrained self-correction architectures (INTERACSPARQL and related tool-augmented self-correction papers) influenced our bounded repair policy. For validator design and conservative static checks we followed prior work on formal/SMT-style policy verification (Bagpipe / Weitz et al., 2016) and incremental network verification techniques. Unlike broad model leaderboards, this study isolates an operational question—marginal value of a single bounded repair call—under a frozen generator, prompt, and validator to provide actionable evidence for pipeline designers.

III. METHODOLOGY

Preregistration and estimand. The preregistration (study_hosted_netops_gvr_v1_cnd_7a3f2b1e; SHA-256 above) specified a paired design where tasks are the unit of pairing. For each task the pipeline (frozen prompt + declared generator snapshot gpt-5-mini; execution/model_configuration.json SHA-256 = 1acce925...) produced a single shared initial candidate. Baseline outcome = validator(initial) (no further model calls). Guarded outcome = validator(initial) OR, if initial failed, a single

automatic repair LLM call is allowed and the repaired candidate revalidated. The primary estimand is the `paired_success_rate_difference_guarded_minus_baseline`.

The preregistration fixed seeds and analysis choices (exact McNemar two-sided test; paired bootstrap percentile 95% CI with 10,000 resamples; seed = 1729). All elements of the experimental contract—task manifest, validator, repair prompt template, stopping rules, retry policy, and analysis code—were frozen and archived prior to model calls.

Task manifest and validator. We used a curated 300-task multi-vendor manifest limited to vendor CLI fragments that express concrete configuration change intents (Cisco IOS, JunOS, Arista fragments). Each manifest entry encodes: initial network state, intent, required safe sequence constraints (e.g., down-before-change, make-before-break), the expected final state, and provenance. The deterministic validator (`deterministic_netops_validator_v5`, v5.0) implements conservative checks: CLI parsing and canonicalization; syntactic correctness; interface existence and assignment invariants; workflow invariants (e.g., must-be-down rules); and destructive pattern detection. Validator outputs are structured traces that include canonicalized final configurations, violation codes, parsed command counts, touched fields, and a boolean pass/fail verdict. Representative scoring artifacts are archived: `execution/scoring/task-000001-baseline.json` (SHA-256 = `1c3cbb30cb9d03...`).

Generator, prompt, and repair. The declared generator snapshot was `gpt-5-mini`; exact model configuration and limits are archived (`execution/model_configuration.json` SHA-256 above). Prompts and the repair-prompt template were frozen and archived. The experiment used a shared initial per task (single generator call). Under the guarded arm the repair call was permitted only if the initial candidate failed validation; only one repair attempt was allowed by design. All provider call traces are archived under `execution/provider_calls/` and cached call artifacts under `execution/call_cache/`.

Execution and reproducibility. The run executed under the declared adapter family hosted `netops_gvr_v1`. All raw outputs, provider traces, scoring artifacts, and hashes are archived and enumerated in `execution/execution_manifest.json`. Key artifact paths and canonical SHA-256 values are referenced in the Results and Disclosure sections below to allow exact replay.

Statistical analysis. The preregistered primary test is an exact McNemar two-sided test on the paired 2×2 contingency table ($n_{11}, n_{10}, n_{01}, n_{00}$). A paired bootstrap percentile 95% CI for the paired difference was computed with 10,000 resamples and seed = 1729. Secondary preregistered analyses included per-vendor paired comparisons with Bonferroni correction (family alpha = 0.05 $\rightarrow$ per-comparison alpha $\approx$ 0.0167). All analysis inputs and outputs are archived (`analysis/paired_contingency_table.csv` SHA-256 = `a39fd3386c94...`).

IV. RESULTS

Execution bookkeeping and raw outcomes. The run executed exactly as preregistered. Planned episodes = 600 (300 tasks $\times$ 2 arms using a shared initial); completed episodes = 600; failed episodes = 0. Model calls used = 300 (one shared initial generation per task); repair calls permitted but not invoked (`repair_call_count` = 0). All execution artifacts and hashes are archived (`execution/execution_manifest.json` lists every artifact and hash). Representative artifact examples: `execution/task_manifest.jsonl` (SHA-256 = `6d99060659a722e6a91dd2abd7baca45a2aefb41e124b049124d5ea2632cb4...`), `execution/model_configuration.json` (SHA-256 = `1acce92558ed...`), and `execution/responses/task-000001-shared-initial.txt` (SHA-256 = `8f38c8be...`).

Primary confirmatory outcome (artifact-grounded). Every shared initial candidate passed the deterministic validator without repair: `baseline_success_count` = 300/300; `guarded_success_count` = 300/300. The paired contingency table is $n_{11} = 300, n_{10} = 0, n_{01} = 0, n_{00} = 0$ (`analysis/paired_contingency_table.csv` SHA-256 = `a39fd3386c94...`). Exact McNemar two-sided $p = 1.00$. Paired bootstrap percentile 95% CI = [0.00, 0.00] (10,000 resamples, seed = 1729). Because no initial failures occurred, the preregistered single-repair policy was never invoked and `repair_call_count` remained zero. Complete accounting (`execution_manifest` SHA-256 listed above) verifies the above counts.

Per-vendor preregistered secondary tests. The preregistration included per-vendor paired tests. We executed exactly as specified and discovered zero discordant pairs overall; consequently each vendor subgroup also exhibits zero discordant pairs and the per-vendor McNemar tests are non-informative under this observed ceiling. Auditors can inspect vendor labels and counts in the canonical task manifest (`execution/task_manifest.jsonl` SHA-256 above) and replay alternative generator prompts or snapshots to generate discordance where needed.

Representative episode diagnostics. Validator traces provide structured diagnostics: `parsed_command_count`, `required_command_count`, `observed_touched_field_count`, `canonicalized_final_configuration`, and violation codes. Example (artifact): `execution/scoring/task-000001-baseline.json` (SHA-256 = `1c3cbb30cb9d0...`) records `parsed_command_count` = 4, `required_command_count` = 4, and `valid` = true with an explicit canonical `final_configuration`. Three representative episodes (`task-000001..000003`) are included in the artifact bundle with identical shared initial / baseline / guarded outputs that passed the validator (artifact paths and hashes are archived).

V. DISCUSSION

Operational interpretation. The study's shared-initial paired design isolates the incremental value of a bounded repair step applied to the exact same generator output. Under the observed generator snapshot, prompt, and task manifest the experiment encountered a ceiling effect: all shared initial candidates already satisfied the conservative validator. In that

regime a bounded single repair cannot increase validator pass probability because no initial failures existed. The statistical result (paired difference = 0.00; exact McNemar $p = 1.00$; CI = [0.00,0.00]) is therefore an accurate and artifact-grounded reflection of the observed data.

Implications for pipeline design. The null outcome advises caution: adding bounded automatic repair increases runtime and call budget only when generator error rates produce validator failures on the target intent distribution. System designers should empirically evaluate whether generator errors occur on their deployment distribution before incurring the operational cost of a repair stage. If generator error rates are near zero, repairs yield no deterministic pass-rate benefit under the bounded policy evaluated here.

Why the ceiling may have occurred. Several non-exclusive explanations are testable from archived artifacts: (1) the chosen task manifest difficulty is modest relative to generator capability; (2) the prompt and few-shot scaffolding used for generation favored validator-compliant outputs; (3) the specific model snapshot (gpt-5-mini) and its alignment tuning achieved high task accuracy. Each hypothesis can be explored by controlled replays: (a) increase task difficulty or adversariality, (b) use independent initial draws per arm (instead of shared initial) to measure repair value under generator variability, (c) permit iterative repair budget >1 , or (d) change generator snapshot. All required artifacts (raw responses, provider traces, prompts, manifest, and validator) are archived with canonical hashes to enable exact reproducibility.

Threats to validity. Internal validity: the shared-initial design controls generator variability but reduces sensitivity to generator variance and can lead to ceiling effects when the generator is already accurate on the manifest. Execution determinism is supported by archived seeds and provider traces (execution/provider_calls/*). Construct validity: the deterministic validator is intentionally conservative (syntactic, interface and workflow invariants) and omits dynamic emulation and some routing semantics; some safety failures detectable only by emulation are not measured here. External validity: the experiment used a single model snapshot, a single prompt family, and a curated 300-task manifest; results do not generalize across models, prompts, or more adversarial task banks. Conclusion validity: zero discordance makes significance testing uninformative for demonstrating repair benefit; however the observed null is a valid experimental outcome and accurately reports that the bounded repair stage produced no additional validator passes under these conditions.

Reproducibility and next steps. We archived the complete execution bundle and canonical SHA-256 hashes (execution/execution_manifest.json). Reproduction consists of replaying the generator and validator against the manifest using the archived prompts and the declared generator identifier; all provider call traces and raw responses are available (execution/provider_calls/ and execution/responses/). Recommended controlled follow-ups enabled by the archive: (1) rerun with independent (non-shared) initial draws per arm to measure repair effect when generator variability exists; (2) increase task

difficulty or append adversarial constraints to the manifest; (3) allow iterative repair attempts or human-in-the-loop repair; (4) evaluate other generator snapshots. Because all artifacts are hash-signed, auditors can perform such experiments and compare outcomes to the preregistered run without ambiguity.

VI. LIMITATIONS

- Single model snapshot (gpt-5-mini) evaluated — results do not generalize to other models or model versions.
- Deterministic validator implements a conservative subset of operational hazards (syntactic checks, interface and workflow invariants) and omits dynamic/emulation-level checks and deeper routing semantics.
- Curated 300-task manifest may not include adversarial or out-of-distribution failure modes; under the chosen prompt and model snapshot every shared initial candidate passed the validator (ceiling effect), limiting the ability to measure G-V-R improvements under the preregistered one-repair policy.
- Shared initial-candidate design (preregistered) fixes the same initial generation for both arms to isolate repair effect; this reduces sensitivity to generator variability and can mask repair value when the generator is already accurate on the task set.
- Only a single automatic repair attempt was permitted per task; iterative or human-in-the-loop repair policies were not evaluated.
- Automated contamination detection cannot eliminate memorization risk entirely; the preregistered contamination summary reported zero flagged pairs but residual uncertainty remains.

VII. CONCLUSION

We preregistered and executed a paired confirmatory experiment to measure whether a bounded single automatic repair improves the deterministic validator pass probability for LLM-generated vendor CLI candidates when the same initial candidate is shared across arms. Under the frozen manifest, prompt, and generator snapshot (gpt-5-mini), every shared initial passed a conservative deterministic validator and the preregistered single-repair policy was never invoked. The paired difference is therefore exactly 0.00 ($n_{11} = 300$, $n_{10} = n_{01} = n_{00} = 0$; exact McNemar two-sided $p = 1.00$; paired bootstrap 95% CI = [0.00,0.00]). This artifact-grounded null result means that, for the tested regime, adding a bounded repair stage does not increase deterministic pass rates. However, the outcome is narrowly contextual: repair utility depends on generator error rates for the target distribution. We release the full preregistration, manifest, raw responses, provider traces, validator traces, and analysis artifacts with canonical SHA-256 hashes to enable exact reproduction and targeted follow-ups that vary task difficulty, model snapshot, prompt, and repair budget. All artifact paths and representative hashes are listed in the execution manifest for auditors and operators to reuse.

DISCLOSURE STATEMENT

Mandatory Disclosure — archived initial master prompt reference and execution provenance: Initial master prompt immutable archived path: provenance/master_prompt.txt; SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba. Human role and autonomy boundary: the Research Director selected the topic family (Generative AI / LLMs for NetOps) and permitted pre-lock infrastructure development; after the autonomy lock the AI autonomously performed literature review, research-gap selection, preregistration design, code generation, experiment execution, analysis, internal AI peer review, manuscript drafting, and artifact hashing. Models and snapshots: declared generator snapshot = gpt-5-mini (execution/model_configuration.json; SHA-256 = 1acce92558edeb13cd97b75817329d332afe4a36d01d566f1a26c61b132923fa). Adapter and execution contract: adapter_family = hosted_netops_gvr_v1; execution_mode = scientific_confirmatory; planned episodes = 600; completed episodes = 600; model_calls_used = 300 (one shared initial generation per task). Preregistration: study_hosted_netops_gvr_v1_cnd_7a3f2b1e (preregistration SHA-256 = a85d66d4c6ce2275e2d8c820f96c610532dbb5452bebb3fe108848f9dc38cdf6). Deterministic validator: deterministic_netops_validator_v5 (validator_version = 5.0); representative validator traces archived (e.g., execution/scoring/task-000001-baseline.json SHA-256 = 1c3cbb30cb9d0...). The exact initial prompting templates, repair prompt templates, and the canonical task manifest (execution/task_manifest.jsonl SHA-256 = 6d99060659a722e6a91dd2abd7baca45a2aefb41e124b049124d5ea2632cb4b3) were frozen prior to model calls and are archived. All raw outputs, provider traces, scoring artifacts, and analysis inputs/outputs are archived and hash-signed in execution/execution_manifest.json to enable exact reproduction. No human manual editing of technical content occurred after the autonomy lock. Technical restarts beyond the preregistered retry policy did not occur. Pre-lock development artifacts were used only to build the autonomous infrastructure and were not imported as empirical results for this final run.

REFERENCES

- [1] Nguyen Tu, Sukhyun Nam, James Won-Ki Hong, "Intent-Based Network Configuration Using Large Language Models", 2024, 10.1002/nem.2313
- [2] Supratim Chakraborty, Nithin Chitta, Rajesh Sundaresan, "Automation of Network Configuration Generation using Large Language Models", 2024, 10.23919/cnsm62983.2024.10814407
- [3] Rajdeep Mondal, Nikolaj Björner, Todd Millstein, Alan Tang, George Varghese, "Tackling Ambiguity in User Intent for LLM-based Network Configuration Synthesis", 2025, 10.1145/3772356.3772402
- [4] Konstantin Weitz, Doug Woos, Emina Torlak, Michael D. Ernst, Arvind Krishnamurthy, Zachary Tatlock, "Scalable verification of border gateway protocol configurations with an SMT solver", 2016, 10.1145/2983990.2984012
- [5] Kaan Aykurt, Andreas Blenk, Wolfgang Kellerer, "NetLLMBench: A Benchmark Framework for Large Language Models in Network Configuration Tasks", 2024, 10.1109/nfv-sdn61811.2024.10807499
- [6] <http://arxiv.org/abs/2307.09288>